

R28

GUIADO EN CLASE

Cliente/servidor TCP y sniffer

Programación para Ciberseguridad · Fase III · AD2026

Un socket es el punto final de una comunicación de red. Un socket crudo (*raw socket*) da acceso a los paquetes antes de que el sistema operativo los procese. Estos dos primitivos son la base de la mayoría de las herramientas de red que existen.

Objetivo: Implementar un cliente TCP, un servidor TCP básico, y un sniffer que decodifica encabezados IP usando únicamente la biblioteca estándar de Python.

Instrucciones

1. Servidor TCP en `r28_servidor.py`:

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.bind(("0.0.0.0", 9999))
servidor.listen(1)
print("Escuchando en puerto 9999...")
cliente, addr = servidor.accept()
print(f"Conexion desde {addr}")
cliente.send(b"Hola desde el servidor\n")
cliente.close()
```

2. Cliente TCP en `r28_cliente.py`:

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 9999))
print(c.recv(1024).decode())
c.close()
```

3. Ejecuta el servidor en una terminal y el cliente en otra. Captura el handshake.
4. **Sniffer** (requiere root/admin) en `r28_sniffer.py`:

```
import socket, struct

s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_TCP)
print("Capturando (Ctrl+C para detener)...")
for _ in range(5):
    paquete, _ = s.recvfrom(65535)
    hdr = struct.unpack('!BBHHBBH4s4s', paquete[:20])
    print(f"Origen: {socket.inet_ntoa(hdr[8])} Destino: {socket.inet_ntoa(hdr[9])}")
    print(f"Proto: {hdr[6]}")
```

Evidencias requeridas

- Screenshot del servidor y cliente corriendo simultáneamente en dos terminales.
- Screenshot del sniffer capturando paquetes con origen y destino visibles.

Preguntas de reflexión

1. Este patrón cliente/servidor es exactamente el de una herramienta de C2 (*Command and Control*). ¿Qué habría que agregar al servidor para que pudiera ejecutar comandos en la máquina del “cliente” remotamente?
2. Los raw sockets requieren permisos de administrador. ¿Por qué el sistema operativo impone esa restricción y no deja que cualquier proceso los use?
3. `struct.unpack` decodifica bytes según una especificación de formato. ¿Qué significa el `!` al inicio del formato `'!BBHHBBH4s4s'`?